

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 4: The Intelligent Catalog and API Blueprint

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO2
Session	Practical 4	Date	

Aim

To enable pgvector in bookflow_db, add a VECTOR(3) embedding column to Books, insert mock theme vectors and find the three books closest to a search vector with the <-> (L2) and <=> (cosine) operators, and to design the OpenAPI contract for GET /v1/books/search with q and limit parameters, a 200 response schema and a 400 error envelope that follows REST constraints.

Program

Part A - p4_vector.sql

```
-- Experiment 4 (Part A): Semantic search with pgvector
\c bookflow_db

-- Task 1: Enable pgvector and add an embedding column
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE Books ADD COLUMN embedding VECTOR(3);

-- Task 2: Mock 3-dimensional theme vectors
-- dimension 1 = space / science fiction, 2 = computer science, 3 = self-improvement
UPDATE Books SET embedding = '[0.95, 0.10, 0.05]' WHERE isbn = '9780553418026'; -- The Martian
UPDATE Books SET embedding = '[0.85, 0.05, 0.10]' WHERE isbn = '9780441172719'; -- Dune
UPDATE Books SET embedding = '[0.92, 0.15, 0.08]' WHERE isbn = '9780593135204'; -- Project Hail Mary
UPDATE Books SET embedding = '[0.05, 0.95, 0.10]' WHERE isbn = '9780132350884'; -- Clean Code
UPDATE Books SET embedding = '[0.02, 0.90, 0.05]' WHERE isbn = '9780078022159'; -- Database System Concepts
UPDATE Books SET embedding = '[0.03, 0.92, 0.04]' WHERE isbn = '9781449373320'; -- Designing Data-Intensive
  Applications
UPDATE Books SET embedding = '[0.10, 0.80, 0.20]' WHERE isbn = '9780135957059'; -- The Pragmatic Programmer
UPDATE Books SET embedding = '[0.02, 0.97, 0.03]' WHERE isbn = '9780262046305'; -- Introduction to Algorithms
UPDATE Books SET embedding = '[0.05, 0.10, 0.95]' WHERE isbn = '9780735211292'; -- Atomic Habits
UPDATE Books SET embedding = '[0.05, 0.20, 0.90]' WHERE isbn = '9781455586691'; -- Deep Work

SELECT book_id, title, embedding FROM Books ORDER BY book_id;

-- Search vector for the query "Stories about space"
-- L2 (Euclidean) distance: smaller value = more similar
SELECT title,
  ROUND((embedding <-> '[0.9, 0.1, 0.1]')::NUMERIC, 4) AS l2_distance
FROM Books
ORDER BY embedding <-> '[0.9, 0.1, 0.1]'
LIMIT 3;

-- Cosine distance (1 - cosine similarity): smaller value = more similar
SELECT title,
  ROUND((embedding <=> '[0.9, 0.1, 0.1]')::NUMERIC, 4) AS cosine_distance,
  ROUND((1 - (embedding <=> '[0.9, 0.1, 0.1]')::NUMERIC, 4) AS cosine_similarity
FROM Books
ORDER BY embedding <=> '[0.9, 0.1, 0.1]'
LIMIT 3;

-- Second search vector for the query "How to write better software"
SELECT title,
  ROUND((embedding <=> '[0.1, 0.8, 0.2]')::NUMERIC, 4) AS cosine_distance
FROM Books
ORDER BY embedding <=> '[0.1, 0.8, 0.2]'
LIMIT 3;
```

Part B - openapi.yaml (designed in Swagger Editor)

```
openapi: 3.0.3
info:
  title: BookFlow Catalog API
  version: 1.0.0
  description: >
```

```

Contract for the BookFlow catalog search service. The API is stateless:
every request carries all the information needed to serve it, and no
session data is stored on the server.
servers:
- url: http://localhost:8000
  description: Local development server

paths:
/v1/books/search:
  get:
    summary: Search the catalog (keyword and semantic search)
    operationId: searchBooks
    tags: [Catalog]
    parameters:
      - name: q
        in: query
        required: true
        description: Free-text search query, for example "stories about space".
        schema:
          type: string
          minLength: 2
          maxLength: 200
      - name: limit
        in: query
        required: false
        description: Maximum number of results to return.
        schema:
          type: integer
          minimum: 1
          maximum: 50
          default: 10
    responses:
      '200':
        description: Matching books ordered by similarity (most similar first).
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SearchResponse'
            example:
              query: stories about space
              count: 2
              results:
                - book_id: 7
                  title: Dune
                  isbn: '9780441172719'
                  published_year: 2019
                  similarity: 0.9986
                - book_id: 10
                  title: Project Hail Mary
                  isbn: '9780593135204'
                  published_year: 2021
                  similarity: 0.9984
      '400':
        description: The request parameters are invalid.
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorEnvelope'
            example:
              error:
                code: INVALID_PARAMETER
                message: Query parameter 'limit' must be between 1 and 50.
                field: limit
                timestamp: '2026-10-02T10:15:30Z'

/v1/books:
  post:
    summary: Create a new book (POST is used for creating resources)
    operationId: createBook
    tags: [Catalog]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/BookCreate'
    responses:
      '201':

```

```

    description: Book created.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Book'
  '400':
    description: Validation failed.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorEnvelope'

components:
  schemas:
    Book:
      type: object
      required: [book_id, title, isbn]
      properties:
        book_id: { type: integer, example: 10 }
        title: { type: string, example: Project Hail Mary }
        isbn: { type: string, minLength: 13, maxLength: 13, example: '9780593135204' }
        published_year: { type: integer, maximum: 2026, example: 2021 }

    BookCreate:
      type: object
      required: [title, isbn]
      properties:
        title: { type: string, minLength: 1 }
        isbn: { type: string, minLength: 13, maxLength: 13 }
        published_year: { type: integer, maximum: 2026 }

    SearchResult:
      allOf:
        - $ref: '#/components/schemas/Book'
        - type: object
          required: [similarity]
          properties:
            similarity:
              type: number
              format: float
              minimum: 0
              maximum: 1
              description: Cosine similarity between the query and the book embedding.

    SearchResponse:
      type: object
      required: [query, count, results]
      properties:
        query: { type: string }
        count: { type: integer, minimum: 0 }
        results:
          type: array
          items:
            $ref: '#/components/schemas/SearchResult'

    ErrorEnvelope:
      type: object
      required: [error]
      properties:
        error:
          type: object
          required: [code, message]
          properties:
            code: { type: string, example: INVALID_PARAMETER }
            message: { type: string }
            field: { type: string, nullable: true }
            timestamp: { type: string, format: date-time }

```

Output

Part A - psql -f p4_vector.sql

```

CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION
ALTER TABLE Books ADD COLUMN embedding VECTOR(3);
ALTER TABLE
UPDATE Books SET embedding = '[0.95, 0.10, 0.05]' WHERE isbn = '9780553418026';
UPDATE 1
UPDATE Books SET embedding = '[0.85, 0.05, 0.10]' WHERE isbn = '9780441172719';

```

```

UPDATE 1
UPDATE Books SET embedding = '[0.92, 0.15, 0.08]' WHERE isbn = '9780593135204';
UPDATE 1
UPDATE Books SET embedding = '[0.05, 0.95, 0.10]' WHERE isbn = '9780132350884';
UPDATE 1
UPDATE Books SET embedding = '[0.02, 0.90, 0.05]' WHERE isbn = '9780078022159';
UPDATE 1
UPDATE Books SET embedding = '[0.03, 0.92, 0.04]' WHERE isbn = '9781449373320';
UPDATE 1
UPDATE Books SET embedding = '[0.10, 0.80, 0.20]' WHERE isbn = '9780135957059';
UPDATE 1
UPDATE Books SET embedding = '[0.02, 0.97, 0.03]' WHERE isbn = '9780262046305';
UPDATE 1
UPDATE Books SET embedding = '[0.05, 0.10, 0.95]' WHERE isbn = '9780735211292';
UPDATE 1
UPDATE Books SET embedding = '[0.05, 0.20, 0.90]' WHERE isbn = '9781455586691';
UPDATE 1

```

```
SELECT book_id, title, embedding FROM Books ORDER BY book_id;
```

book_id	title	embedding
1	Clean Code	[0.05,0.95,0.1]
2	Database System Concepts	[0.02,0.9,0.05]
3	The Martian	[0.95,0.1,0.05]
7	Dune	[0.85,0.05,0.1]
8	Atomic Habits	[0.05,0.1,0.95]
9	Designing Data-Intensive Applications	[0.03,0.92,0.04]
10	Project Hail Mary	[0.92,0.15,0.08]
11	The Pragmatic Programmer	[0.1,0.8,0.2]
12	Deep Work	[0.05,0.2,0.9]
13	Introduction to Algorithms	[0.02,0.97,0.03]

(10 rows)

```

SELECT title,
       ROUND((embedding <-> '[0.9, 0.1, 0.1]')::NUMERIC, 4) AS l2_distance
FROM   Books
ORDER  BY embedding <-> '[0.9, 0.1, 0.1]'
LIMIT  3;

```

title	l2_distance
Project Hail Mary	0.0574
Dune	0.0707
The Martian	0.0707

(3 rows)

```

SELECT title,
       ROUND((embedding <=> '[0.9, 0.1, 0.1]')::NUMERIC, 4) AS cosine_distance,
       ROUND((1 - (embedding <=> '[0.9, 0.1, 0.1]'))::NUMERIC, 4) AS cosine_similarity
FROM   Books
ORDER  BY embedding <=> '[0.9, 0.1, 0.1]'
LIMIT  3;

```

title	cosine_distance	cosine_similarity
Dune	0.0014	0.9986
Project Hail Mary	0.0016	0.9984
The Martian	0.0017	0.9983

(3 rows)

```

SELECT title,
       ROUND((embedding <=> '[0.1, 0.8, 0.2]')::NUMERIC, 4) AS cosine_distance
FROM   Books
ORDER  BY embedding <=> '[0.1, 0.8, 0.2]'
LIMIT  3;

```

title	cosine_distance
The Pragmatic Programmer	0.0000
Clean Code	0.0121
Database System Concepts	0.0226

(3 rows)

Part B - specification validation

```

$ openapi-spec-validator openapi.yaml
openapi.yaml: OK

```

REST constraint verification for the designed contract

```

GET  /v1/books/search -> safe, read-only fetch; parameters q (required) and limit (1-50, default 10)
POST /v1/books       -> creates a resource, returns 201 Created
200 OK               -> SearchResponse {query, count, results[]} ordered by similarity
400 Bad Request      -> ErrorEnvelope {error: {code, message, field, timestamp}}

```

Statelessness	-> no cookies or session parameters; every request is self-contained
Uniform interface	-> resource nouns (/books), versioned path (/v1), JSON media type

Result

For the 'stories about space' vector [0.9, 0.1, 0.1], both L2 and cosine distance returned Project Hail Mary, Dune and The Martian, none of which contain the word 'space' in the title, and the software query returned The Pragmatic Programmer, Clean Code and Database System Concepts. The OpenAPI 3.0.3 contract validated without errors and follows statelessness and correct HTTP verbs (GET to fetch, POST to create).